

1.5 — Apply Agent SDK Hooks for Tool Call Interception and Data Normalization

Concept

The Agent SDK exposes a hook system that lets you attach callbacks at key points in the agentic loop without modifying the core execution logic. Two hooks are central to this: `beforeToolCall` fires immediately after Claude requests a tool but before the tool executes, and `afterToolCall` fires after the tool returns but before the result is sent back to the model. These hooks receive the full tool call context — name, input arguments, and (in the case of `afterToolCall`) the raw output — giving you a structured intercept point for validation, transformation, and logging.

This architecture matters because it separates cross-cutting concerns from tool implementation. A tool that queries a database should not need to know about audit logging, input sanitization, or output schema normalization. Hooks handle those responsibilities once, centrally, and consistently. Every tool call passes through the same pipeline regardless of which tool is invoked or which part of your codebase registered it.

The distinction between **deterministic compliance** and **probabilistic compliance** is critical here. Probabilistic compliance means adding instructions to the system prompt such as "always return dates in ISO 8601 format" and hoping the model follows them. This works most of the time but fails silently when it doesn't. Deterministic compliance means enforcing the rule in code — in an `afterToolCall` hook that parses and reformats every date field in every tool output before the model ever sees it. The hook runs every time, unconditionally. For anything that must be reliably true — sanitized inputs, normalized schemas, redacted PII, validated ranges — deterministic enforcement via hooks is the correct approach.

Code Example

```
import Anthropic from "@anthropic-ai/sdk";

const client = new Anthropic();

// --- Hook definitions ---

function beforeToolCall(toolName: string, toolInput: Record<string, unknown>): Record<string, unknown> {
  // Validate that required fields are present
  if (toolName === "query_database" && typeof toolInput.table !== "string") {
    throw new Error(`beforeToolCall: 'table' must be a string, got ${typeof toolInput.table}`);
  }
}
```

```

// Normalize inputs deterministically – no model instruction needed
const normalized = { ...toolInput };
if (typeof normalized.limit === "number") {
    normalized.limit = Math.min(normalized.limit, 100); // enforce hard cap
}

return normalized;
}

function afterToolCall(
    toolName: string,
    rawOutput: unknown
): Record<string, unknown> {
    if (typeof rawOutput !== "object" || rawOutput === null) {
        return { result: rawOutput };
    }

    const output = rawOutput as Record<string, unknown>;

    // Normalize date fields to ISO 8601 – deterministic, not instructional
    const normalized = Object.fromEntries(
        Object.entries(output).map(([key, value]) => {
            if (key.endsWith("_at") || key.endsWith("_date")) {
                const parsed = new Date(value as string);
                return [key, isNaN(parsed.getTime()) ? value : parsed.toISOString()];
            }
            return [key, value];
        })
    );

    // Redact sensitive fields before they reach the model context
    const { password, secret, api_key, ...safe } = normalized as Record<string, unknown>;
    void password; void secret; void api_key; // explicitly discarded

    return safe;
}

// --- Tool executor that applies hooks ---

async function executeToolWithHooks(
    toolName: string,
    rawInput: Record<string, unknown>,
    tools: Record<string, (input: Record<string, unknown>) => Promise<unknown>>
): Promise<Record<string, unknown>> {
    const validatedInput = beforeToolCall(toolName, rawInput);

    const tool = tools[toolName];

```

```

    if (!tool) throw new Error(`Unknown tool: ${toolName}`);

    const rawOutput = await tool(validatedInput);
    return afterToolCall(toolName, rawOutput);
}

// --- Usage in the agentic loop ---

async function runAgentWithHooks(userMessage: string) {
    const tools: Anthropic.Tool[] = [
        {
            name: "query_database",
            description: "Query a database table",
            input_schema: {
                type: "object" as const,
                properties: {
                    table: { type: "string" },
                    limit: { type: "number" },
                },
                required: ["table"],
            },
        },
    ],
    ];

    const toolImpls: Record<string, (input: Record<string, unknown>) => Promise<unknown>> = {
        query_database: async (input) => ({
            rows: [{ id: 1, created_at: "2024-01-15", password: "hunter2" }],
            table: input.table,
        }),
    };

    const messages: Anthropic.MessageParam[] = [
        { role: "user", content: userMessage },
    ];

    while (true) {
        const response = await client.messages.create({
            model: "claude-sonnet-4-5",
            max_tokens: 1024,
            tools,
            messages,
        });

        if (response.stop_reason === "end_turn") {
            return response;
        }
    }
}

```

```

const toolResults: Anthropic.ToolResultBlockParam[] = [];

for (const block of response.content) {
  if (block.type === "tool_use") {
    const normalizedOutput = await executeToolWithHooks(
      block.name,
      block.input as Record<string, unknown>,
      toolImpls
    );
    toolResults.push({
      type: "tool_result",
      tool_use_id: block.id,
      content: JSON.stringify(normalizedOutput),
    });
  }
}

messages.push({ role: "assistant", content: response.content });
messages.push({ role: "user", content: toolResults });
}
}

```

Common Mistakes

1. **Using system prompt instructions instead of hooks for data contracts.** Writing "always return dates as ISO 8601" in the system prompt is probabilistic — the model may comply, but it is not guaranteed. If downstream systems depend on date format consistency, enforce it in `afterToolCall` where the transformation is unconditional.
2. **Mutating the input object passed to `beforeToolCall`.** Modifying the original input in-place creates hidden side effects and makes debugging difficult. Always return a new object (`{ ...toolInput, limit: cap }`) rather than reassigning properties on the argument directly.
3. **Swallowing hook errors silently.** A `beforeToolCall` hook that catches a validation failure and returns a default value instead of throwing allows invalid tool calls to proceed undetected. Throw explicitly with a descriptive message so the failure surfaces in logs and halts execution at the correct point.

References

- [Anthropic Agent SDK — Hooks](#)
- [Tool Use Overview](#)
- [Building Effective Agents](#)